資訊工程導論
第六章:程式語言(Programming Languages)

本講義未經同意請勿自行翻印

教科書: Computer Science an overview, 13th Edition, J. Glenn Brookshear and D. Brylow, Pearson, Addison Wesley, Pub. (滄海圖書代理) 中英文版

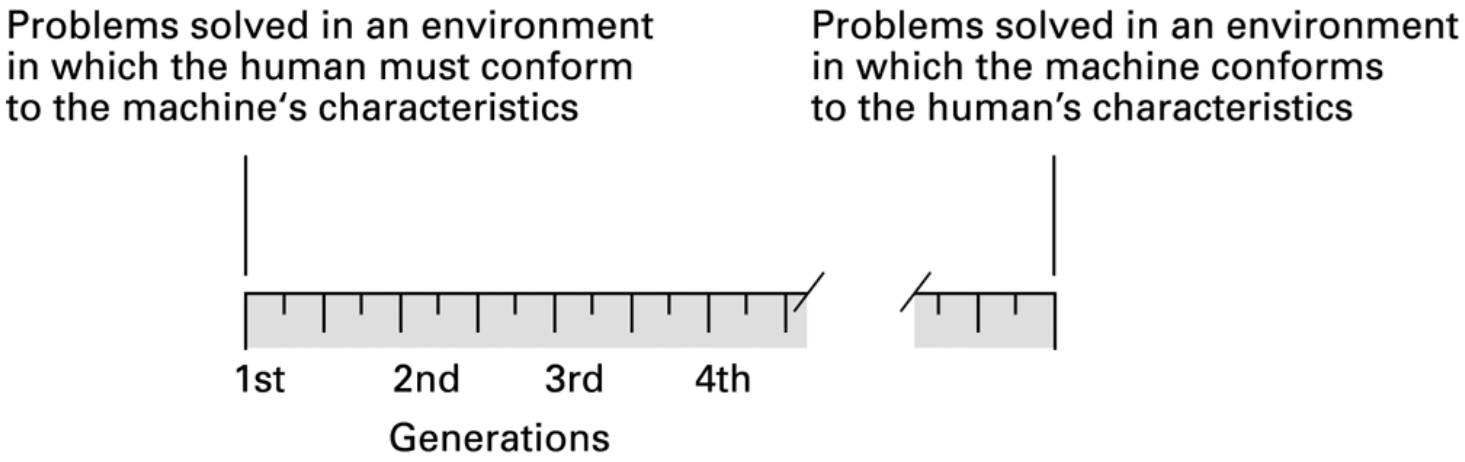
第 6 章 程式語言

- 6.1 歷史觀
- 6.2 傳統程式設計概念
- 6.3 程序單元
- ~~6.4 程式語言實作~~
- 6.5 物件導向程式設計
- 6.6 ~~平行化程序的程式設計~~
- ~~6.7 宣告式程式設計~~

© 滄海圖書

6.1 歷史觀

- 早期程式世代
  - 機器語言(machine language)
  - 組合語言(assembly languae)
- 與機器無關語言
- 後期 – 更強大的抽象化



© 滄海圖書

第二世代：組合語言

- 以通俗表示法(mnemonics:記憶術)來描述機器指令
  - 指令碼的通俗名稱
  - 程式**變數(variables)**或**識別符號(identifiers)**：由程式設計師以描述性名稱來表示記憶體位址

© 滄海圖書

組合語言的特性

- 機器指令與組合指令有一對一的對應關係
  - 程式設計師必須從機器角度來思考
- 本質上具機器相關性
- 由**組譯器(assembler)**將組合語言轉譯成機器語言

© 滄海圖書

程式範例

- 機器語言

```
0x156C
0x166D
0x5056
0x306E
0xC000
```

- 組合語言

```
LD R5, Price
LD R6, ShippingCharge
ADDI R0, R5, R6
ST R0, TotalCost
HLT
```


© 滄海圖書

Example

- Python

print("hello")



- ARM Assembly

```
.section .data
msg:    .asciz "hello"

.section .text
.global _start

_start:
    // Write the message to stdout
    ldr r0, =1          // File descriptor 1 (stdout)
    ldr r1, =msg        // Pointer to message
    ldr r2, =5          // Length of message
    mov r7, #4          // System call number for sys_write
    svc 0               // Invoke the kernel

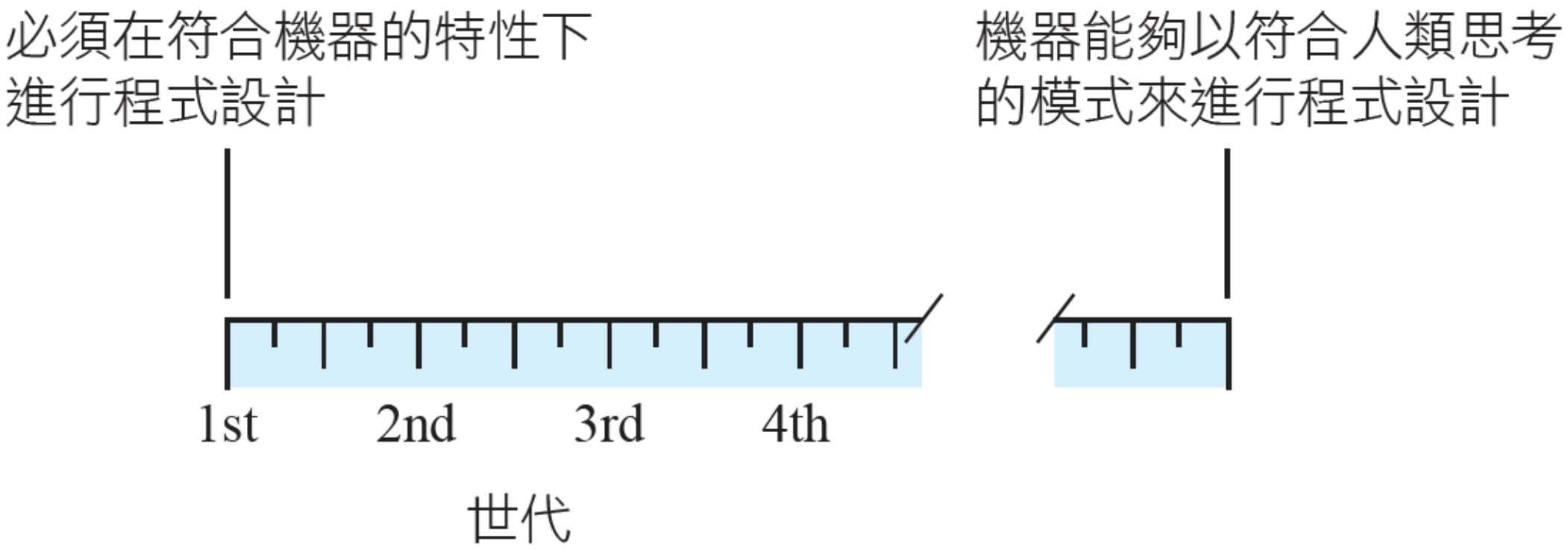
    // Exit the program
    mov r7, #1          // System call number for sys_exit
    mov r0, #0          // Exit status
    svc 0               // Invoke the kernel
```

第三代程式語言

- 使用高階原式
  - 類似第五章所介紹的虛擬碼
- 大多數與機器無關
- 例如：FORTRAN, COBOL
- 每個原式相當於一組機器語言指令
- 由**編譯器**將程式轉換成機器語言

© 滄海圖書

圖6.1 程式語言的世代



© 滄海圖書

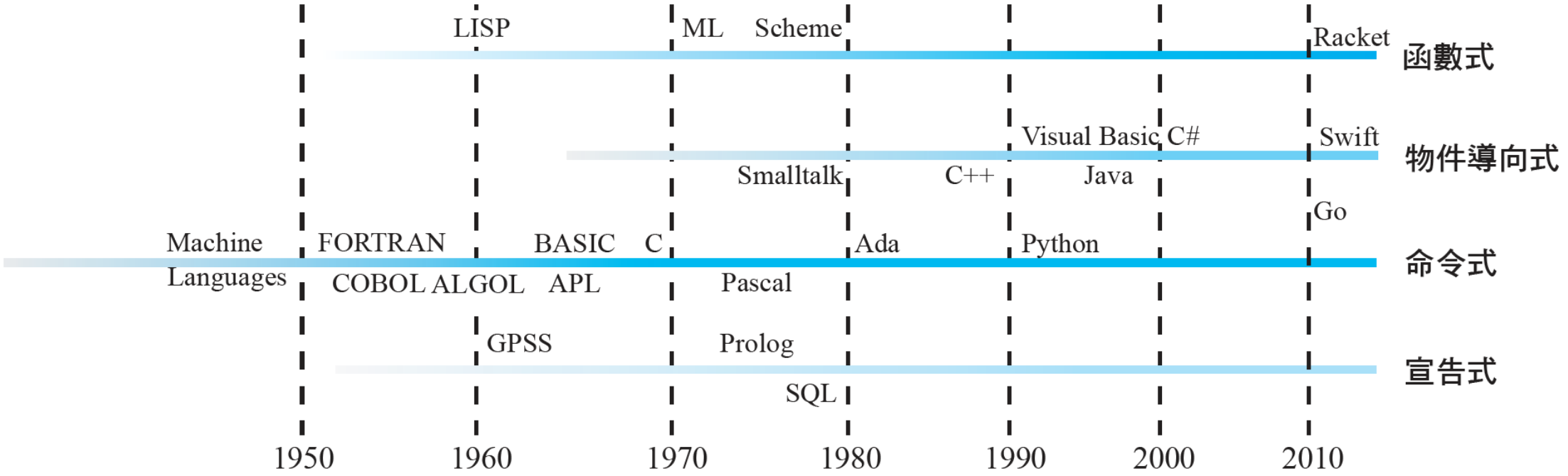
Programming Paradigm

- Functional paradigm(函數式程式法)
- Object-oriented paradigm(物件導向程式法)
- imperative paradigm(命令式程式法)
  - or called procedural paradigm
- declarative paradigm(宣告式程式法)

© 滄海圖書

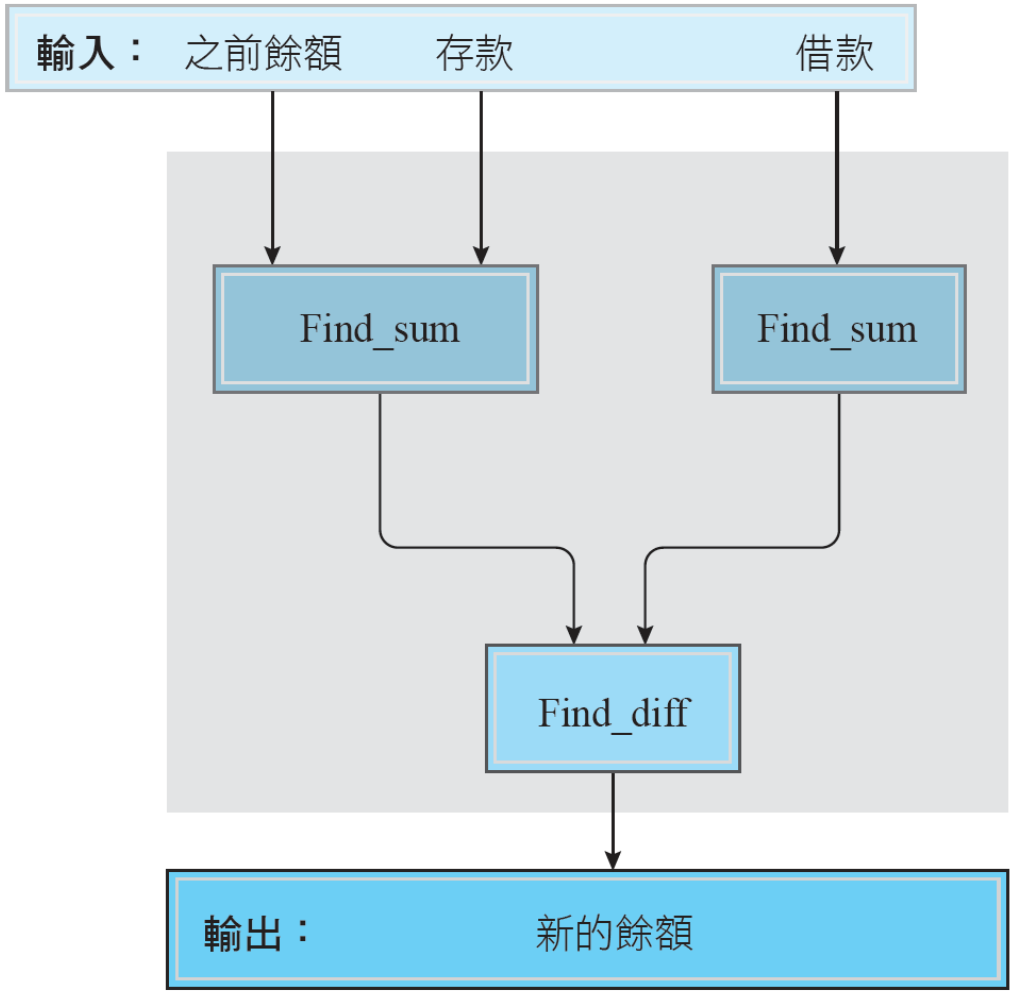


圖6.2 程式設計方法的演進



© 滄海圖書

圖6.3 由兩個簡易函數建構出來的支票簿餘額計算函數



© 滄海圖書

In-class Exercise 3.1

- 命令式程式法可概述為著重於描述導引解決問題的過程，試概述宣告式程式法、函數式程式法與物件導向式程式法。

- What is Cross-Platform Software?
- What are natural languages and formal(正規) languages?

© 滄海圖書

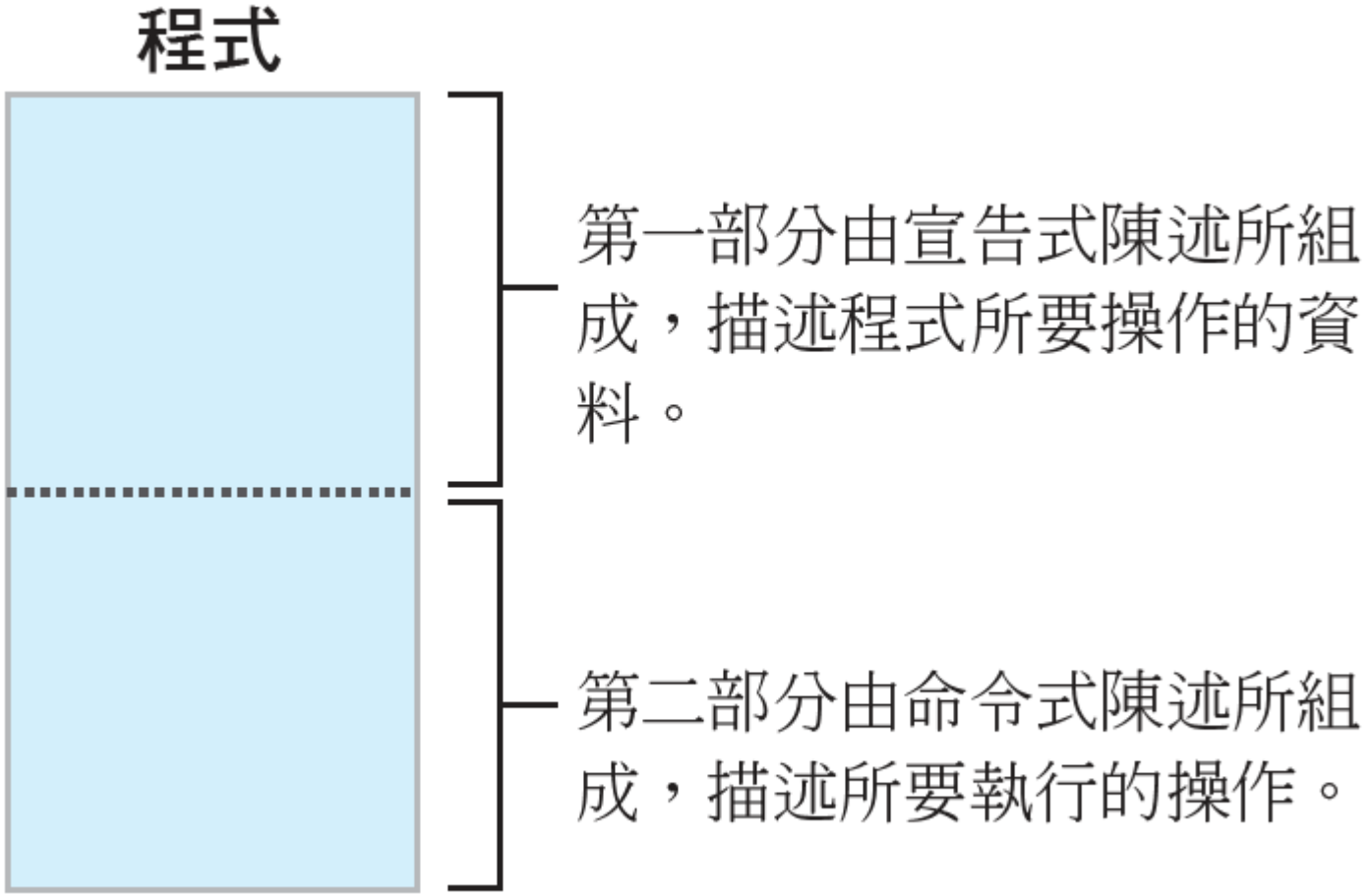


6.2 傳統程式設計概念

- 高階程式語言 (C, C++, Java, C#, FORTRAN) 包含不同種類的抽象化
  - 簡單：常數、定數、變數
  - 複雜：陳述、運算式、控制
  - 深奧：程序、模組、函數庫

© 滄海圖書

圖6.4 一般的命令式程式或程式單元的組成



© 滄海圖書

資料型別(Data Types)

- **整數(Integer)**：所有數字
- **實數 (Float: 浮點數)**：帶有小數的數值
- **字元(Character)**：符號
- **布林(Boolean)**：True/false

© 滄海圖書

變數與資料型別

```
float Length, Width;

int Price, Tax, Total;

char Symbol;
```

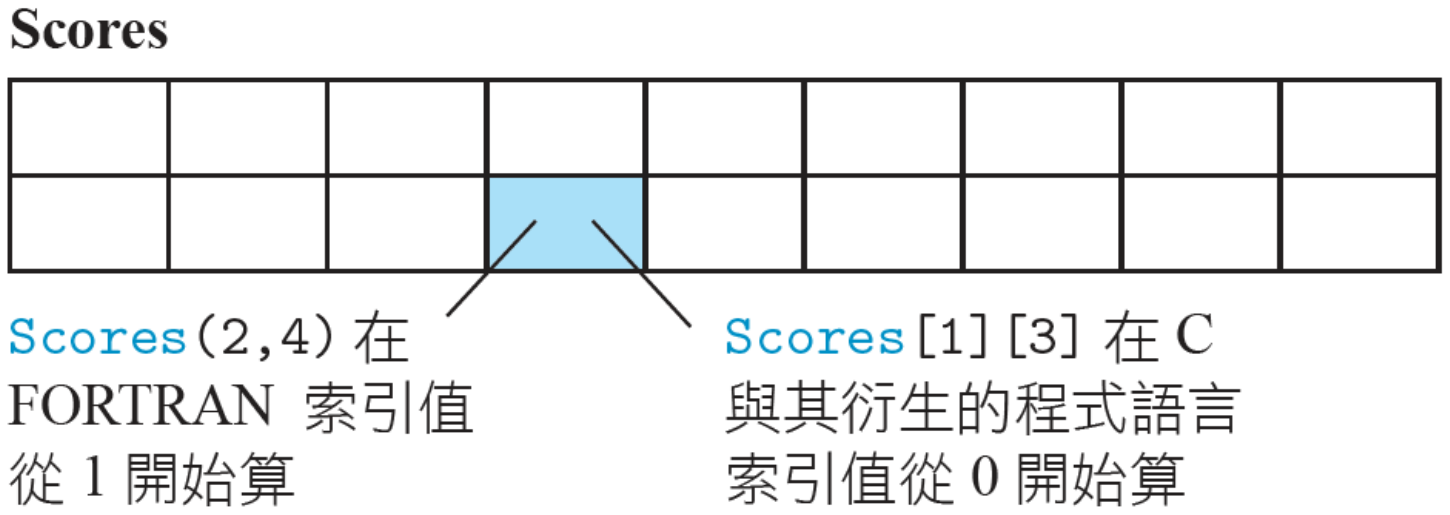

© 滄海圖書

資料結構(Data Structure)

- 資料概念性的組織方式
- 一種常見的資料結構是陣列(array)
  - C 語言　　int Scores[2][9];
  - FORTRAN 語言　　INTEGER Scores(2,9)
- Other:
  - structure(結構)

© 滄海圖書

圖6.5 具有2 列和9 行的二維陣列



- Row-major: loc(a_{ij}) = a + n*(i-1)*d +(j-1)*d;

where a: the start; n: number of elements per row; i: row; j:column

d: 1 bytes per element

© 滄海圖書

Example in Python

- How to initialize the 2D array in Python?

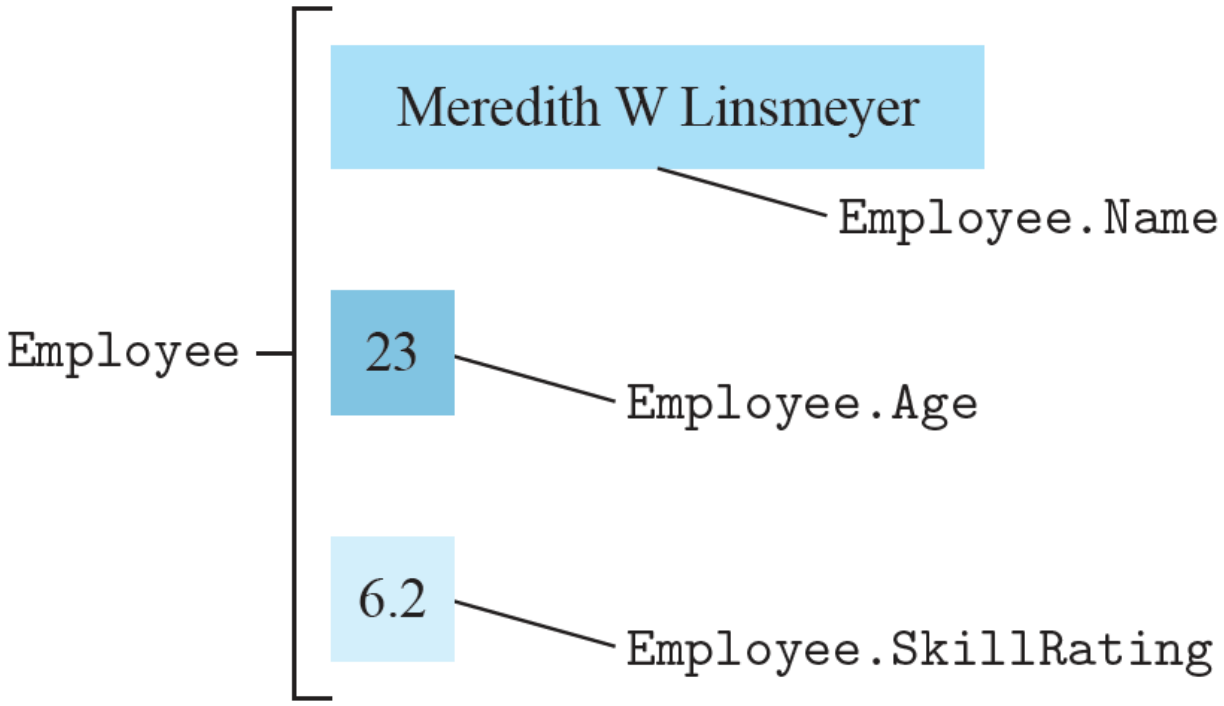
[1]:
```
out = []
for m in range(1, 2+1):
    row = []
    for n in range(1, 9+1):
        row.append(0)
        out.append(row)
print(out)
```

```
[[0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0, 0], [0, 0, 0, 0, 0, 0, 0, 0, 0]]
```

[1]:
```
out = []
for m in range(1, 2+1):
    row = []
    for n in range(1, 9+1):
        row.append(0)
        out.append(row)
print(out)
```

© 滄海圖書

圖6.6 structure Employee 的意象




```
struct Employee{
        string name;
        int age;
        int skill_rating;

};

struct Employee emp;
emp.name;
emp.age;
emp.skill_rating;
```


© 滄海圖書

Example in Python

- What about using structure in Python?

[3]:
```
class Employee:
    def __init__(self, name, age, skill_rating):
        self.name = name
        self.age = age
        self.skill_rating = skill_rating
```

[9]:
```
# Creating an instance of Employee
emp = Employee("Tony", "40", "100")
```

[11]:
```
# Accessing the attributes
print("name:", emp.name)
```

```
name: Tony
```

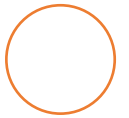
[12]:
```
print("age:", emp.age)
```

```
age: 40
```

[13]:
```
print("skill_rating:", emp.skill_rating)
```

```
skill_rating: 100
```

© 滄海圖書



指定陳述(Assignment Statements)

- 在 C, C++, C#, Java 等語言中

```
Z = X + Y;
```

- 在 Ada 語言中

```
Z := X + Y;
```

- 在 APL (A Programming Language) 語言中

```
Z ← X + Y
```


© 滄海圖書

控制陳述(Control Statements)

- Go to 陳述

```
      goto 40
20    Evade()
      goto 70
40    if (KryptoniteLevel <
          LethalDose) then goto 60
      goto 20
60    RescueDamsel()
70    ...
```

- 使用單一陳述

```
if (KryptoniteLevel < LethalDose):
        RescueDamsel()
else:
        Evade()
```


© 滄海圖書

控制陳述 (If)

- In Python

```
if (condition):
    statementA
else:
    statementB
```

- In C, C++, C#, and Java

```
if (condition) statementA; else
statementB;
```

- In Ada

```
IF condition THEN
    statementA;
ELSE
    statementB;
END IF;
```


© 滄海圖書

控制陳述 (While)

- In Python

```
while (condition):
    body
```

- In C, C++, C#, and Java

```
while (condition) { body }
```

- In Ada

```
WHILE condition LOOP
    body
END LOOP;
```


© 滄海圖書

控制陳述 (Switch)

- In C, C++, C#, and Java

```
switch (variable) {
        case 'A': statementA; break;
        case 'B': statementB; break;
        case 'C': statementC; break;
        default: statementD }
```

- In Ada

```
CASE variable IS
        WHEN 'A'=> statementA;
        WHEN 'B'=> statementB;
        WHEN 'C'=> statementC;
        WHEN OTHERS=>statementD;
END CASE;
```


© 滄海圖書

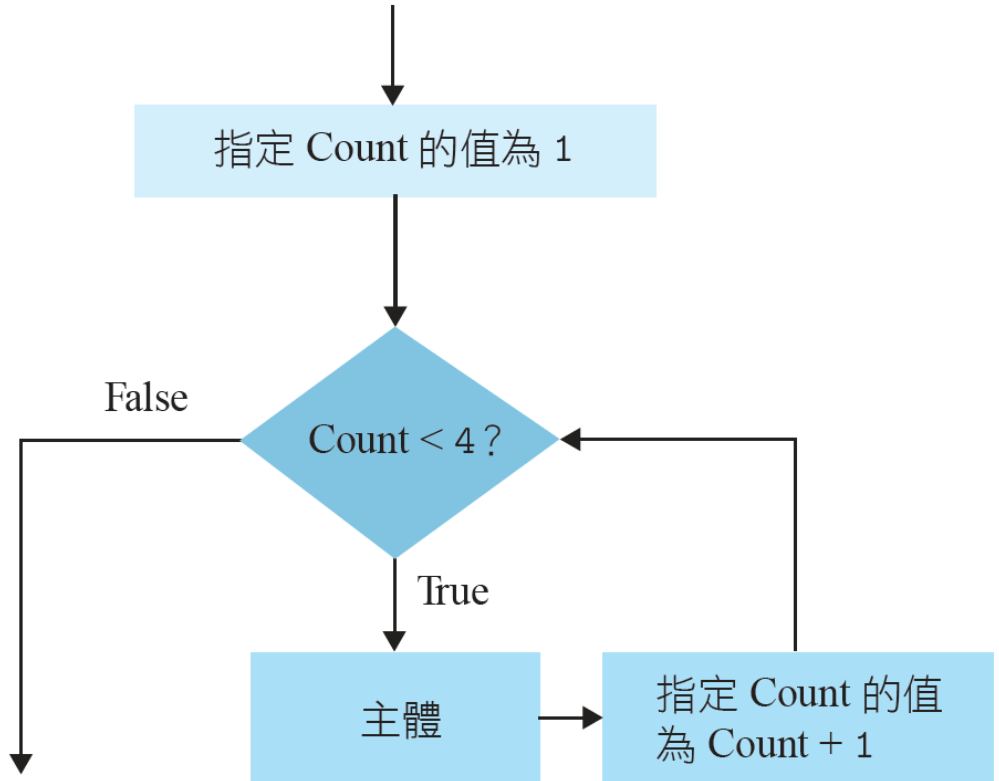
Example in Python

[14]:
```
var = "A"
match var:
    case "A":
        print("A")
    case "B":
        print("B")
    case _:
        print("None")
```

```
A
```

© 滄海圖書

圖6.7 C++, C#, 和Java 語言的for 迴圈結構與其表示法




```
for (int Count = 1; Count < 4; Count++)
    body;
```



```
for count in range(1, 4):
    print(count)
```


© 滄海圖書

註解(Comments)

- 程式中的說明陳述
- 有助於程式的閱讀
- 編譯器會忽略註解

- 在C++, C# 和Java

```
/* This is a comment. */

// This is a comment.
```


© 滄海圖書

Python Comments

- **Single-line Comment**

```
1  # declare a variable
2  name = "John"
3
4  # print name
5  print(name)     # John
```

- **Multiline Comments**

```
1  # This is an example of a multiline comment
2  # created using multiple single-line commenced
3  # The code prints the text Hello World
4  print("Hello, World!")
```

```
1  '''This is an example
2  of multiline comment'''
3  print("Hello, World!")
```

In-Class Exercise 3.2

- 試問陣列與聚合型別有何差異？

© 滄海圖書

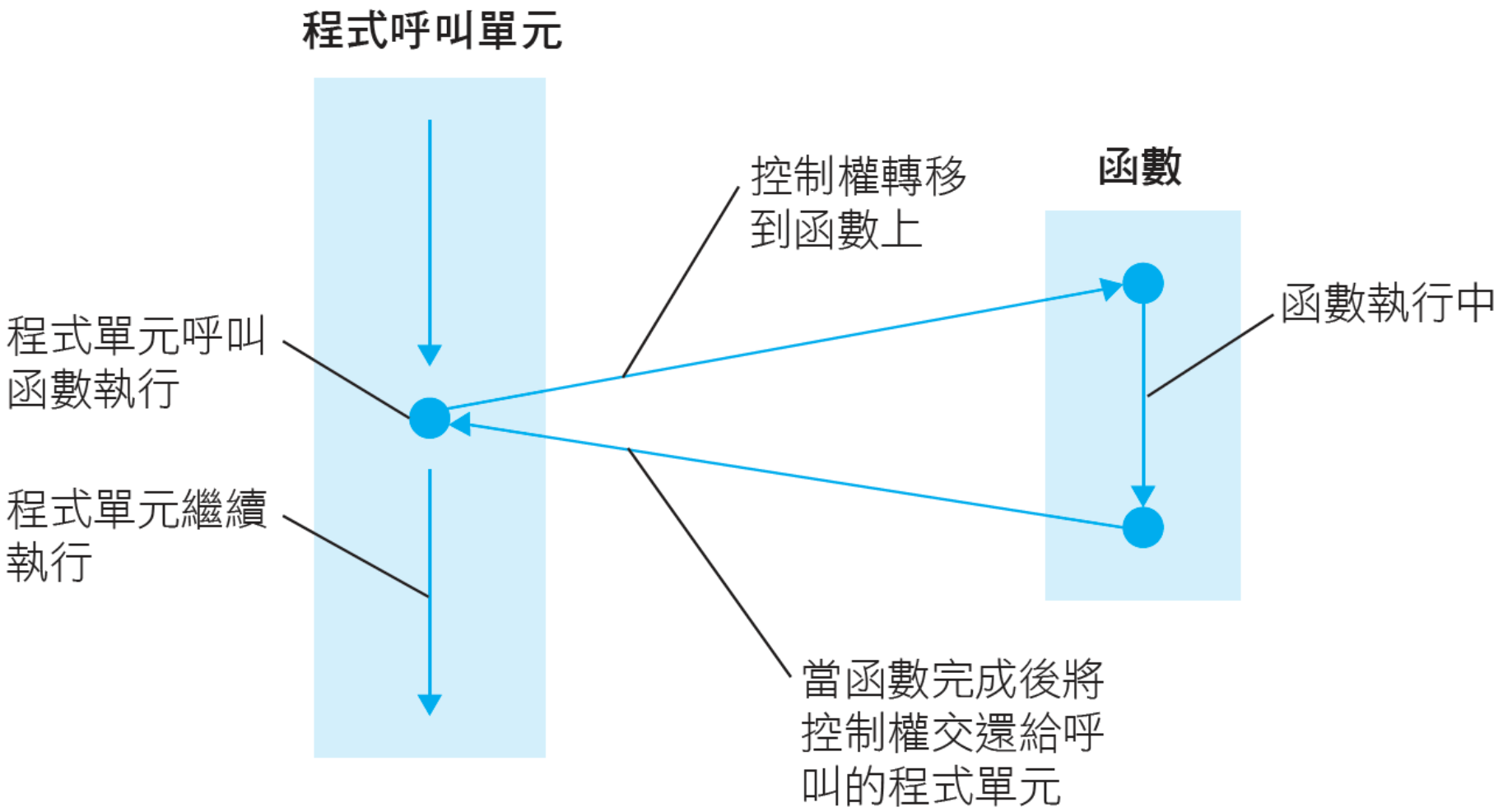


6.3 程序單元(Procedural Units)

- 關於此概念有多種不同名詞：
  - 副程式(Subprogram)、副常式(subroutine) 、程序(procedure)、方法(method,)和函數(function)
- 程序單元始於函數的**標頭(header)**
- 區域(local)變數與全域(global)變數
- 形式參數(formal parameter)與實際參數(actual parameter)
- 定數(literals) v.s 常數(constant)
- 傳值參數(pass by value)與傳址參數(pass by address)

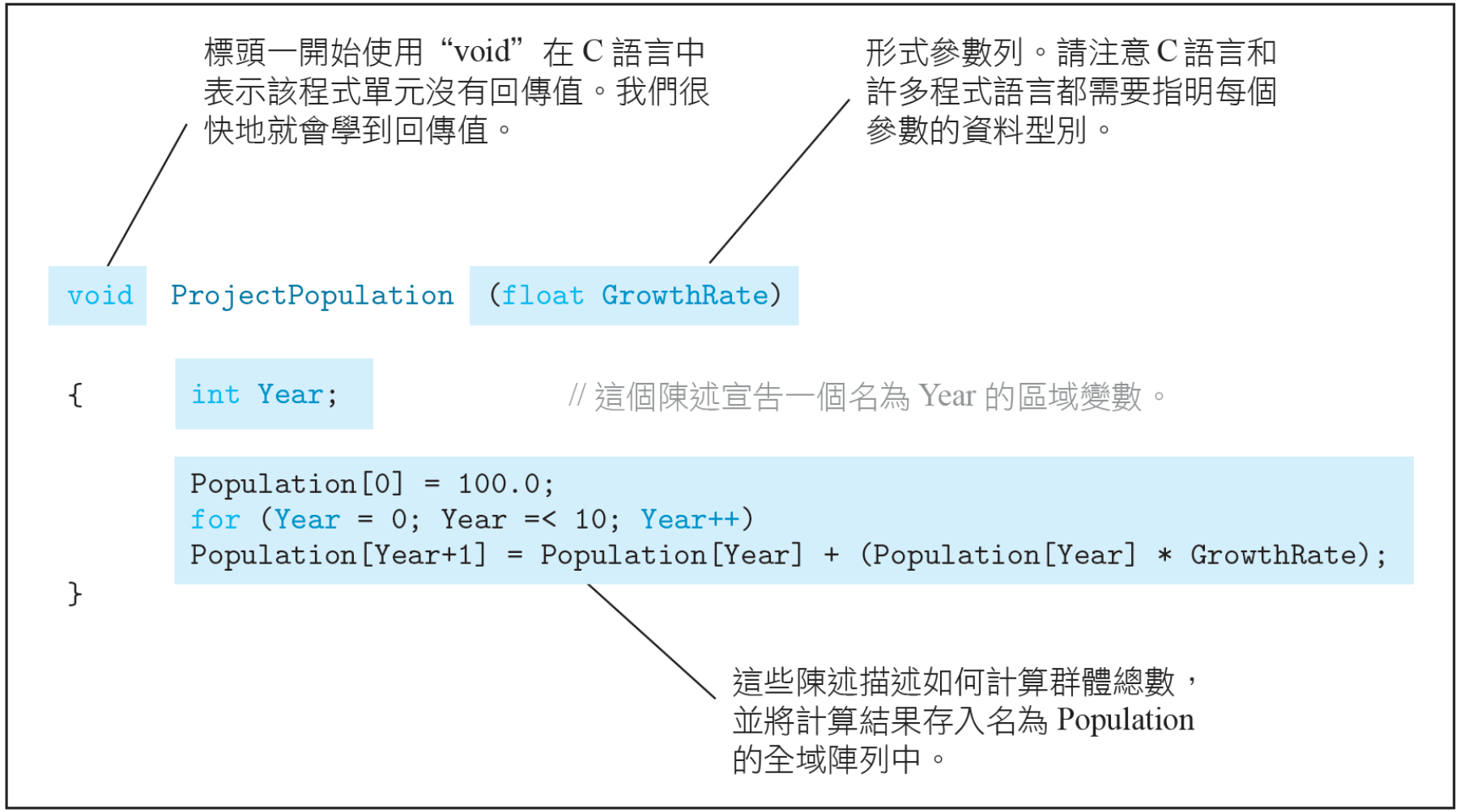
© 滄海圖書

圖6.8 使用函數時的控制流程



© 滄海圖書

圖6.9 使用C 語言所撰寫的函數 ProjectPopulation



© 滄海圖書

Example in Python

```
def ProjectPopulation(growth_rate:float)->float:
    population = [100.0]
    for year in range(0, 10+1):
        population.append(population[year]+(population[year]*growth_rate))
        print(population[year])
    return population[year]
```

```
print(f"The result is: {ProjectPopulation(0.03)}")
```

```
100.0
103.0
106.09
109.2727
112.550881
115.92740743
119.4052296529
122.987386542487
126.67700813876161
130.47731838292447
134.3916379344122
The result is: 130.47731838292447
```

© 滄海圖書

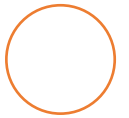
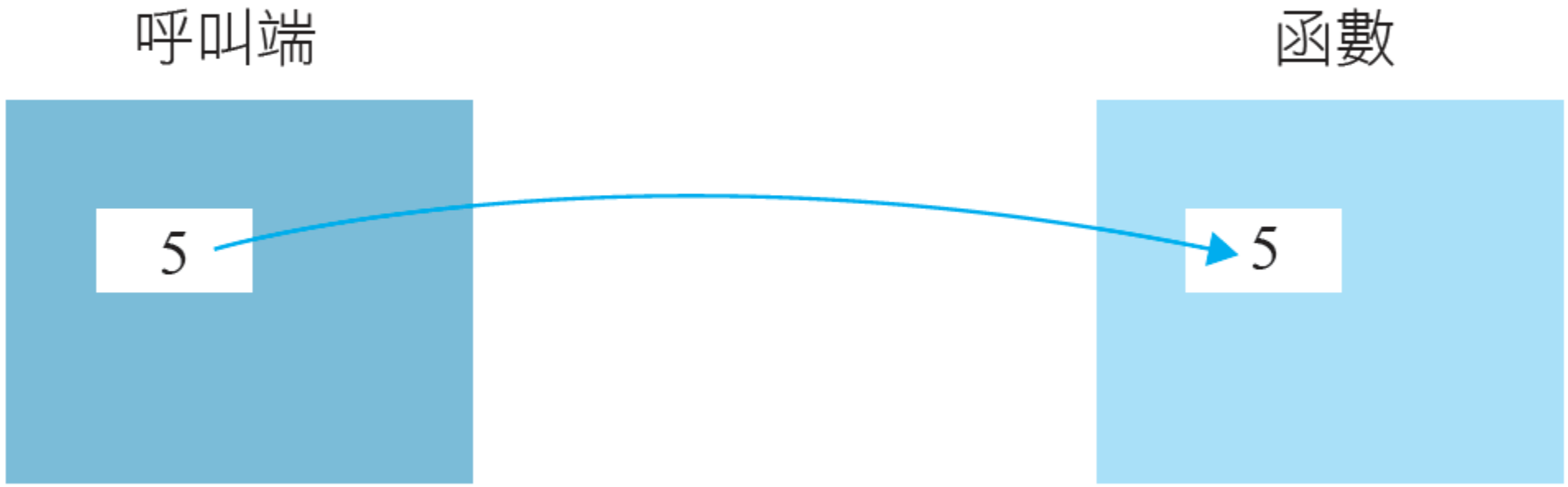


圖6.10 執行函數Demo 並使用傳值參數

a. 當呼叫函數時，會複製一份資料給函數



© 滄海圖書

圖6.10 執行函數Demo 並使用傳值參數 (續)

b. 函數對複製資料進行處理



© 滄海圖書

圖6.10 執行函數Demo 並使用傳值參數 (續)

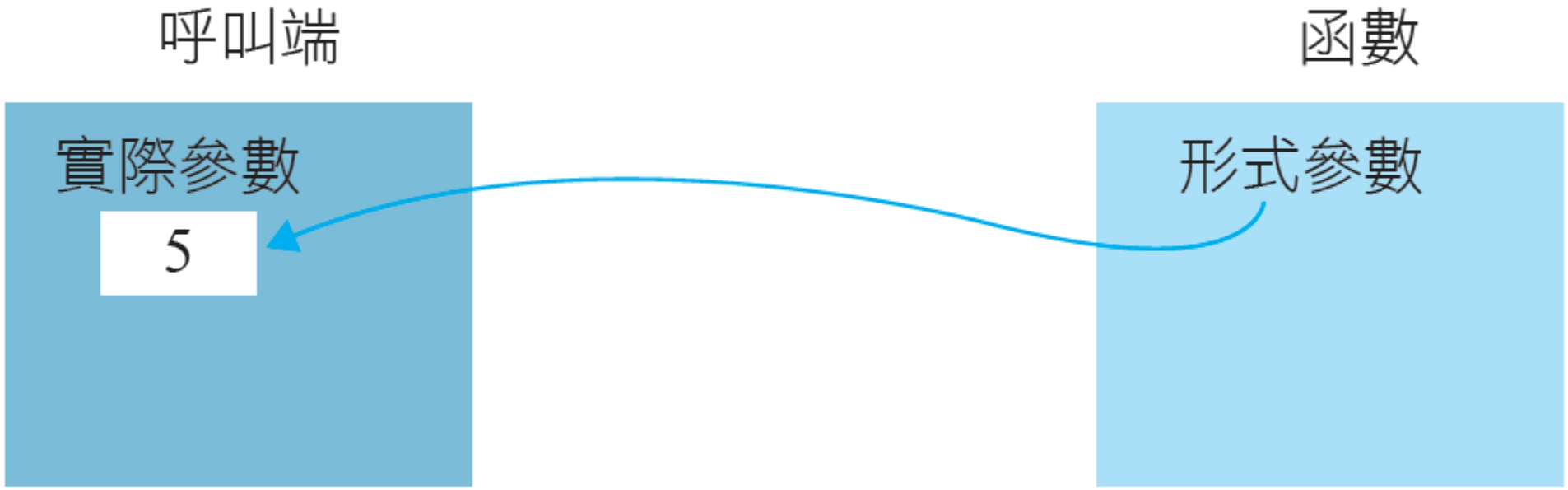
c. 因此當函數結束後，呼叫端的變數並沒有被改變



© 滄海圖書

圖6.11 執行函數Demo 並使用傳址參數

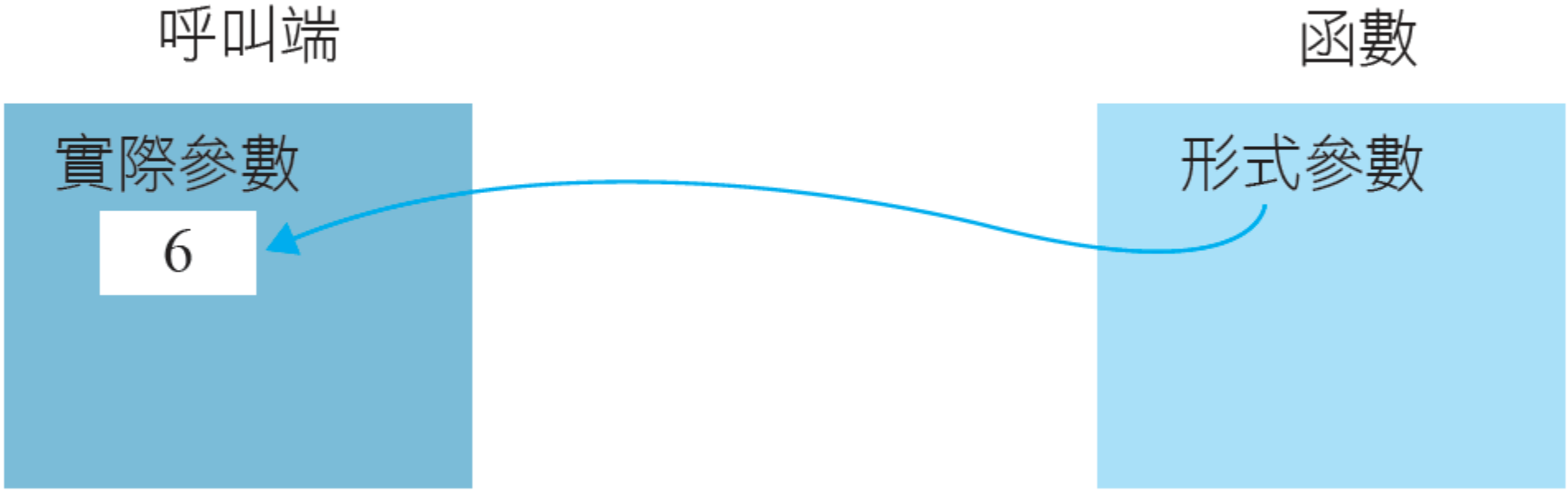
a. 當呼叫函數時，形式參數指向實際參數



© 滄海圖書

圖6.11 執行函數Demo 並使用傳址參數 (續)

b. 因此在函數中所產生的改變會影響實際的參數



© 滄海圖書

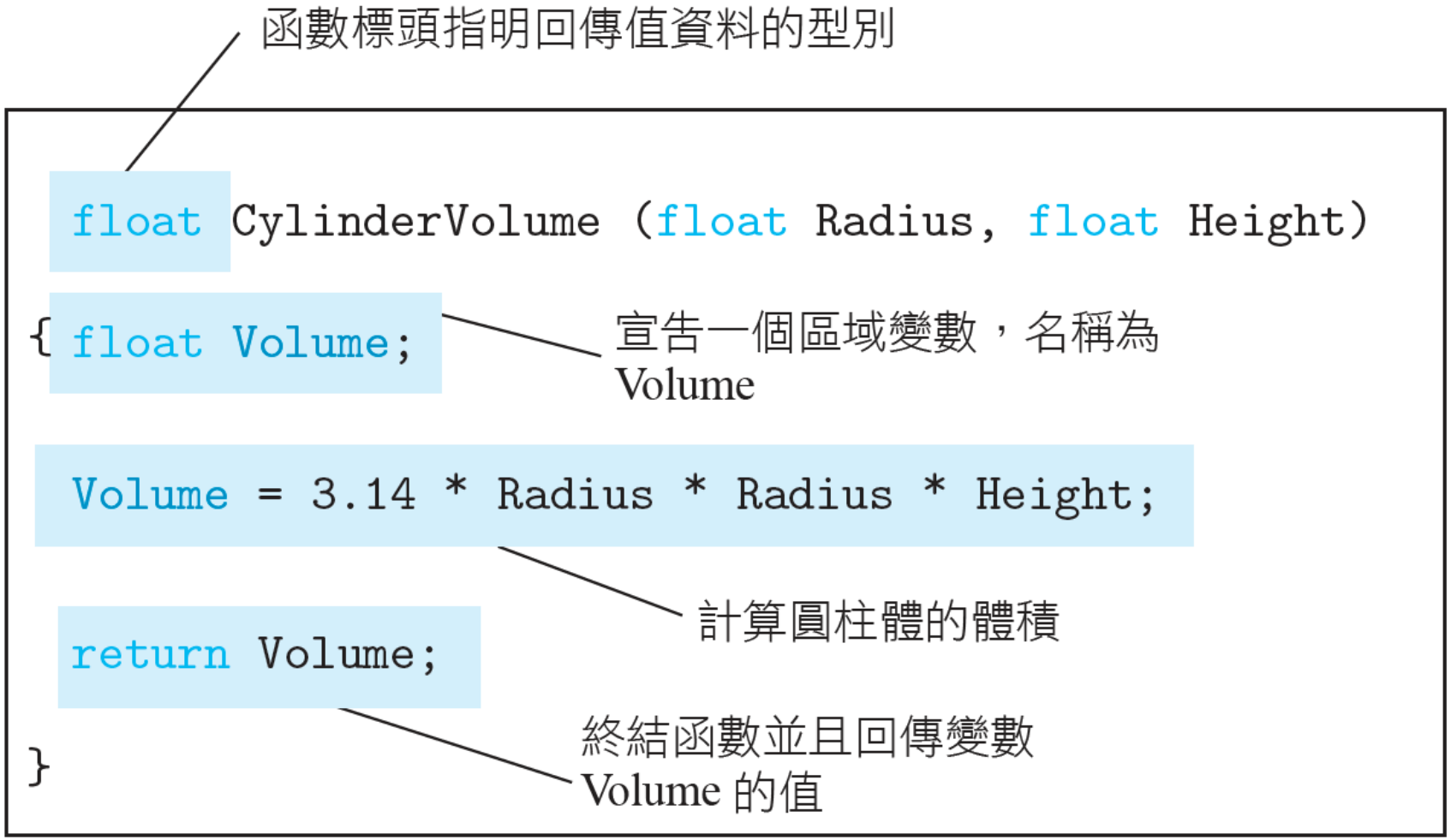
圖6.11 執行函數Demo 並使用傳址參數 (續)

c. 當函數結束後，變動後的結果得以保存下來



© 滄海圖書

圖6.12 以C 語言所撰寫的有回傳值函數 CylinderVolume



© 滄海圖書

Example in Python

[37]:
```
def cylinderVolume(radius: float, height: float)->float:
    vol: float
    vol = 3.14 * radius * radius * height
    return vol
```

[38]:
```
print(f"The result is: {cylinderVolume(3.0, 19.9)}")
```

```
The result is: 562.3739999999999
```

© 滄海圖書



In-Class Exercise 3.3

- 試問傳址參數與傳值參數的函數差異為何？

© 滄海圖書



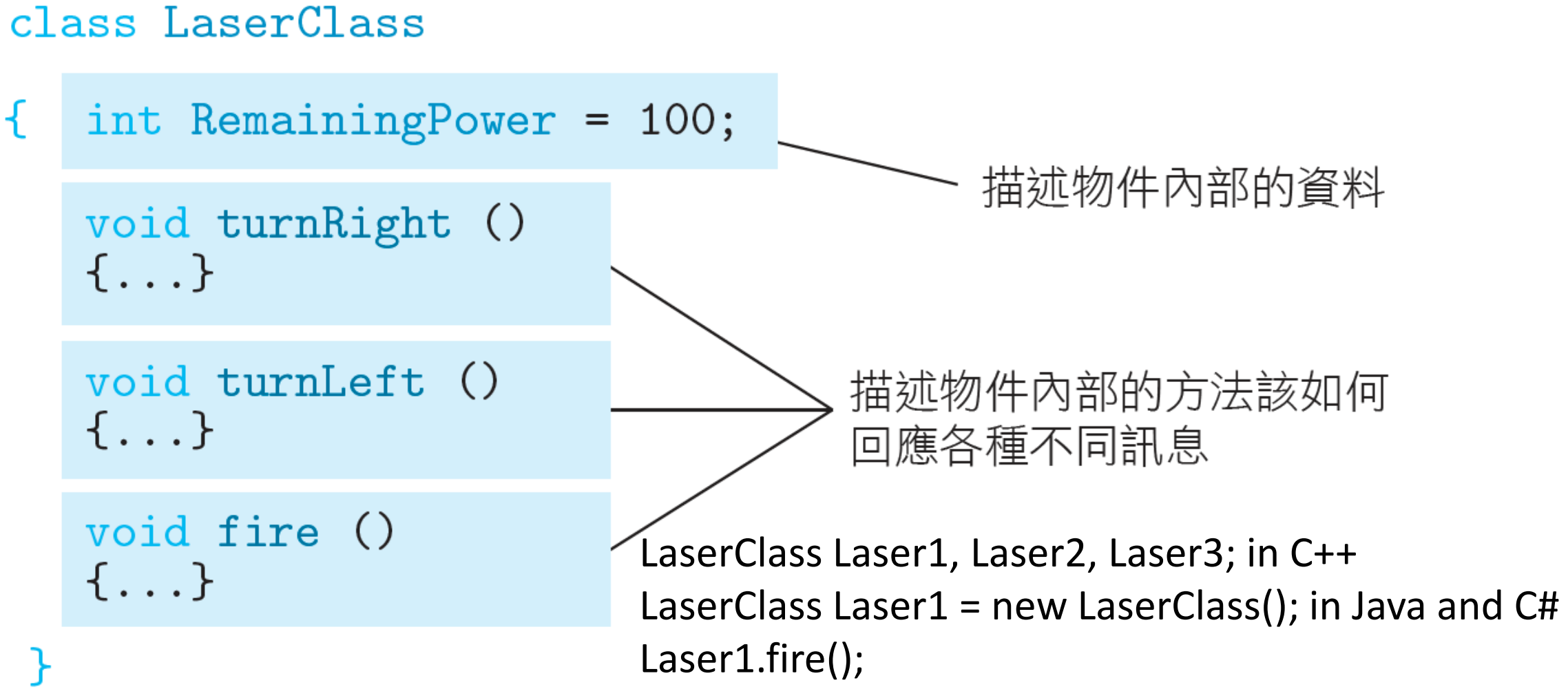
6.5 物件導向程式設計

- **物件(object)**：包含資料與程序的程式單元
- **類別(class)**：建構物件的模版(template)

- 物件可說是類別的**實例(instance)**

© 滄海圖書

圖6.19 描述電腦遊戲中的雷射槍武器的類別結構



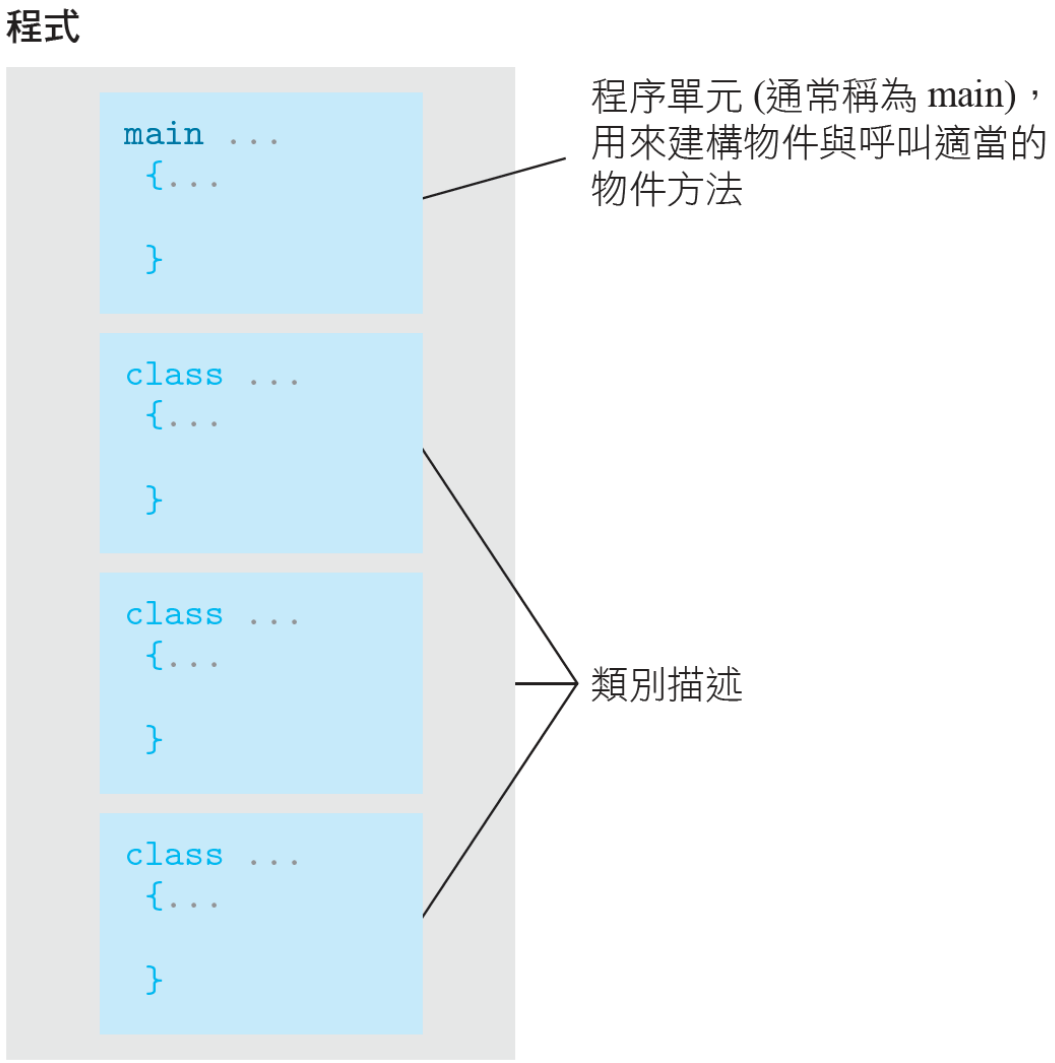
© 滄海圖書

物件的組成要素

- **實例變數(Instance Variable)**：物件中的變數
  - 儲存物件的資訊內容
- **方法(Method)**：物件中的程序
  - 描述物件可執行的動作
- **建構子(Constructor)**：當物件建立時會當先執行的特殊方法，用來初始化新的物件; e.g. __init__()
- **解構子 (Destructor)**:當物件的生命週期結束時，它會自動地被呼叫執行，用來清空並釋放物件先前建立或是佔用的記憶體資源; e.g. __del__()

© 滄海圖書

圖6.20 典型的物件導向式程式的結構



© 滄海圖書

圖6.21 具有建構子的類別

```
LaserClass Laser1 = new LaserClass(50);
LaserClass Laser2 = new LaserClass(100);
```

```
class LaserClass
{ int RemainingPower;
  LaserClass(InitialPower)
  { RemainingPower = InitialPower;
  }
  void turnRight()
  {...}
  void turnLeft()
  {...}
  void fire()
  {...}
}
```

當物件建立時，建構子會指定數值給變數RemainingPower

© 滄海圖書

物件完整性(Integrity)

- **封裝(encapsulation)**：限制物件內部元件的存取方式=>**visibility(能見度)**
  - **私有(private)**
  - **公用(public)**
  - 其他
    - protected
    - package

© 滄海圖書

圖6.22 在Java 或C# 程式中使用封裝來定義 LaserClass 類別

類別中的元件被指定為 public 或 private 取決於它們是否應該讓其他程式單元存取

```
class LaserClass
{private int RemainingPower;
 public LaserClass (InitialPower)
 {RemainingPower = InitialPower;
 }
 public void turnRight ( )
 {...}
 public void turnLeft ( )
 {...}
 public void fire ( )
 {...}
}
```


© 滄海圖書

Class Example in Python

```python
class LaserClass:
    def __init__(self, Name, RemainingPower):
        self.__RemainingPower = RemainingPower
        self.Name = Name

    1 usage
    def turnRight(self):
        print("Turning right")

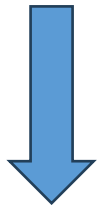
    1 usage
    def turnLeft(self):
        print("Turning left")

    1 usage
    def fire(self):
        print("Firing")

    1 usage
    def readEngery(self):
        print(f"The remainingPower is {self.__RemainingPower}")

    def __del__(self):
        print(f"Laser Gun {self.Name} is deleted")
```

```python
LaserGun = LaserClass( Name: 'Tsai', RemainingPower: 100)
LaserGun.turnRight()
LaserGun.turnLeft()
LaserGun.fire()
LaserGun.readEngery()
```



```
C:\Users\PTsai\AppData\Loca
Turning right
Turning left
Firing
The remainingPower is 100
Laser Gun Tsai is deleted
```

© 滄海圖書



其他物件導向式概念

- **繼承(inheritance)**：讓新的類別可基於既有的類別來定義
  - 基礎類別(base class) or 雙親類別(parent class)=> super().
  - 衍生類別(derived class) or 子類別(child class)
- **多型(polymorphism)**：依據接收函數呼叫的物件類別來引發對應的方法
  1. Overloading(多載): 在相同類別內，定義名稱相同，但引數個數或型態不同的函數，如此程式語言便可依據引數的個數或型態，呼叫相對應的函數。
  2. Overriding(改寫;重載):在子類別裏，定義名稱、引數個數與型態均與父類別相同的函數，同以改寫父類別裡函數的功用。

© 滄海圖書

Inheritance Example in Python

[47]:
```
class Bird():
    def __init__(self, name):
        self.name = name

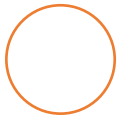

class Sparrow(Bird):
    def __init__(self, name):
        super().__init__('little birdie' + " " + name)
```

[46]:
```
bird = Bird('Bird')
sparrow = Sparrow('Mikey')
```

```
print(bird.name)
print(sparrow.name)
```

```
Bird
little birdie Mikey
```

© 滄海圖書



Polymorphism: Overloading Example in Python

```
def square(x: int)->int:
    return x * x

def square(y: float)->float:
    return y * y
```

```
print(square(7))
print(square(7.5))
```

```
49
56.25
```

© 滄海圖書



...on

```
class Bird():
    def __init__(self, name):
        self.name = name

    def move(self):
        pass

class Sparrow(Bird):
    def __init__(self, name):
        super().__init__('little birdie' + " " + name)

    def move(self):
        return "Hovering"
```

```
bird = Bird('Bird')
sparrow = Sparrow('Mikey')
print(bird.name)
print(sparrow.name)
print(sparrow.move())
```

```
Bird
little birdie Mi
Hovering
```

圖書

In-Class Exercise 3.5

- 試問為何類別中有些元素會被指定為 private ？

© 滄海圖書



總結

- 了解程式語言從機器語言到高階語言的發展，掌握其演進過程及抽象化的重要性（如第三代語言的高階指令）
- 熟悉命令式、物件導向、函數式和宣告式等不同的程式設計範式及其應用場景
- 掌握基本資料型別（如整數、實數、字元、布林）以及常用資料結構（如陣列和結構）
- 瞭解程式中的控制結構，如if-else、while和switch語句，學會應用邏輯控制流程
- 學會建立與使用程序單元（如副程式、函數、方法），理解區域與全域變數、傳值與傳址的差異
- 掌握如何撰寫註解以提升程式碼的可讀性，學會撰寫易於維護的程式
- 理解類別、物件、封裝、繼承、多型等物件導向基礎，並靈活應用建構子與解構子的功能

© 滄海圖書